

1 Theoretische Grundlagen

Dieses Kapitel legt die mathematischen und physikalischen Grundlagen dar, auf denen die Simulationspipeline aufbaut. Das Verständnis dieser Konzepte ist Voraussetzung für die Bewertung der in späteren Kapiteln beschriebenen Implementierungsentscheidungen.

1.1 Akustische Grundlagen

Das Inverse Quadratgesetz Im freien, reflexionsfreien Schallfeld breitet sich Schall kugelförmig vom Emitter aus. Da die abgestrahlte Schallleistung P über eine mit r^2 wachsende Kugeloberfläche $A = 4\pi r^2$ verteilt wird, nimmt die Schallintensität I mit dem Quadrat der Entfernung ab:

$$I(r) = \frac{P}{4\pi r^2}$$

Da die Schallintensität proportional zum Quadrat des Schalldrucks ist ($I \propto p^2$), folgt für den Schalldruck:

$$p(r) \propto \frac{1}{r}$$

Eine Verdopplung des Abstands zur Quelle halbiert den Schalldruck, was einer Pegelminderung von 6 dB entspricht. In der Simulation wird dieser geometrische Energieverlust implizit berücksichtigt: Strahlen, die einen längeren Weg zur Mikrofonkapsel zurücklegen, treffen mit einem entsprechend geringeren Druckbeitrag ein.

Materialabsorption Trifft ein Schallstrahl auf eine Raumgrenze, wird ein Teil der Schallenergie absorbiert. Dieser Vorgang wird durch den dimensionslosen Absorptionskoeffizienten $\alpha \in [0,1]$ beschrieben. Der Restdruck nach einer einzelnen Reflexion berechnet sich zu:

$$p_{\text{ref}} = p_{\text{ein}} \cdot (1 - \alpha)$$

Bei einer Sequenz von n Reflexionen mit den jeweiligen Koeffizienten $\alpha_1, \dots, \alpha_n$ gilt:

$$p_n = p_0 \cdot \prod_{k=1}^n (1 - \alpha_k)$$

Ein vollständig absorbierendes Material ($\alpha = 1,0$) vernichtet den Strahl bei der ersten Reflexion, während ein perfekt reflektierendes Material ($\alpha = 0,0$) keine Energie entzieht. Die Simulation terminiert Strahlen, sobald ihr verbleibender Druckwert den konfigurierbaren Grenzwert `min_pressure` unterschreitet.

Frühe Reflexionen und späte Nachhallzeit Die akustische Impulsantwort eines Raumes lässt sich zeitlich in zwei charakteristische Phasen unterteilen. Innerhalb der ersten ca. 80 ms nach dem Direktschall treffen die *frühen Reflexionen* (Early Reflections) ein: diskrete, noch einzeln wahrnehmbare Echos über klar definierte, kurze Ausbreitungspfade, die das Raumgefühl und die Tiefenwahrnehmung maßgeblich prägen. Mit zunehmender Zeit überlagern sich Reflexionen höherer Ordnung zu einem dichten, statistisch gleichverteilten Nachhallfeld. Diese *späte Nachhallzeit* (Late Reverberation) klingt exponentiell ab und wird durch die Nachhallzeit RT_{60} charakterisiert – der Zeitraum, in dem der Schalldruckpegel um 60 dB abfällt. Das stochastische Raytracing erfasst beide Phasen natürlich, da sowohl kurze Direktpfade als auch komplexe Mehrfachreflexionen in der Impulsantwort akkumuliert werden.

1.2 Monte-Carlo-Integration und Raytracing

Das akustische Integrationsproblem Der Schalldruck am Empfänger ist das Integral über alle möglichen Schallpfade vom Emitter, gewichtet nach Weglänge und materialabhängigen Verlusten. Da die Anzahl dieser Pfade durch Mehrfachreflexionen kombinatorisch explodiert, ist eine analytische Auswertung für beliebige Raumgeometrien nicht praktikabel. Deterministisches Ray Tracing kann zwar exakte Pfade verfolgen, skaliert jedoch exponentiell mit der Anzahl der Reflexionen.

Der Monte-Carlo-Ansatz Die Monte-Carlo-Integration umgeht dieses Problem durch stochastische Stichprobennahme. Statt alle Pfade zu berechnen, schätzt man den Erwartungswert eines Integrals durch den arithmetischen Mittelwert von N zufällig gezogenen Stichproben:

$$\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{i=1}^N f(x_i)$$

Nach dem Gesetz der großen Zahlen konvergiert dieser Schätzer für $N \rightarrow \infty$ gegen den wahren Integralwert. Der statistische Fehler nimmt dabei mit $\frac{1}{\sqrt{N}}$ ab: ein viermal so großes Stichprobenvolumen halbiert den Fehler.

Anwendung: Stochastisches Raytracing In der Simulation entspricht eine Stichprobe einem Schallstrahl, der in einer zufälligen Richtung $\theta \in [0, 2\pi)$ vom Emitter ausgesendet wird:

$$\vec{D} = (\cos \theta, \sin \theta), \quad \theta \sim \mathcal{U}(0, 2\pi)$$

Jeder Strahl verfolgt seinen Ausbreitungspfad, reflektiert an Wänden und registriert einen Treffer, sobald er die Mikrofonkapsel schneidet. Der akkumulierte Beitrag dieses Treffers – Laufzeit τ und Restdruck p – repräsentiert eine Stichprobe des akustischen Integrals. Über $N = 10^6$ bis 10^8 Strahlen konvergiert die Gesamtheit der Treffer zur vollständigen Impulsantwort des Raumes. Der

wesentliche Vorteil gegenüber deterministischen Verfahren liegt in der trivialen Parallelisierbarkeit: Da jeder Strahl unabhängig von allen anderen ist, lassen sich die N Stichproben direkt auf beliebig viele CPU-Kerne aufteilen (siehe Kapitel 3).

1.3 Digitale Signalverarbeitung: Impulsantwort und schnelle Faltung

Die Impulsantwort (IR) Ein lineares, zeitinvariantes System – wie ein Raum aus akustischer Perspektive – ist vollständig durch seine *Impulsantwort* $h(t)$ charakterisiert: die Ausgabe des Systems bei einem idealen Dirac-Impuls $\delta(t)$ am Eingang. Die von der Rust-Engine berechnete Menge an (Laufzeit, Druck)-Paaren $\{(\tau_i, p_i)\}_{i=1}^M$ stellt eine diskrete Approximation dieser Impulsantwort dar:

$$h(t) \approx \sum_{i=1}^M p_i \cdot \delta(t - \tau_i)$$

Das Python-Modul `convolution.py` diskretisiert diese Darstellung in ein samplegenaues Array, indem jede Verzögerung τ_i in den Sample-Index $n_i = \lfloor \tau_i \cdot f_s \rfloor$ umgerechnet und Druckwerte akkumuliert werden (bei $f_s = 44100$ Hz).

Das Faltungsintegral Das Ausgangssignal $y(t)$ – das hallbehaftete Audio – ergibt sich aus der linearen Faltung des Eingangssignals $x(t)$ mit der Impulsantwort $h(t)$:

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(\tau) h(t - \tau) d\tau$$

In der diskreten Signalverarbeitung wird dieses Integral zur Summe:

$$y[n] = \sum_{k=0}^{M-1} h[k] \cdot x[n - k]$$

Die direkte Auswertung hat eine Rechenkomplexität von $\mathcal{O}(N \cdot M)$, wobei N die Länge des Audiosignals und M die Länge der Impulsantwort bezeichnet. Bei realen Impulsantworten mit M von mehreren hunderttausend Samples ist eine direkte Berechnung unpraktikabel.

Schnelle Faltung via FFT (Overlap-Add) Das *Faltungstheorem* der Fouriertransformation besagt, dass eine Faltung im Zeitbereich einem punktweisen Produkt im Frequenzbereich entspricht:

$$Y(f) = \mathcal{F}\{x\}(f) \cdot \mathcal{F}\{h\}(f) \implies y[n] = \mathcal{F}^{-1}\{Y(f)\}$$

Durch den Einsatz der Fast Fourier Transform (FFT) sinkt die Komplexität auf $\mathcal{O}((N + M) \log(N + M))$, was bei großen M einen massiven Speedup bedeutet.

Da das Audiosignal wesentlich länger sein kann als eine einzelne FFT erlaubt, wird das *Overlap-Add*-Verfahren eingesetzt: Das Signal $x[n]$ wird in

überlappungsfreie Blöcke der Länge L aufgeteilt, jeder Block wird separat mit $h[n]$ gefaltet, und die resultierenden Ausgabeblöcke – je $L + M - 1$ Samples lang – werden additiv überlagert. Die Implementierung nutzt hierfür `scipy.signal.oaconvolve`, welche diesen Algorithmus effizient realisiert und sowohl mono- als auch mehrkanalige Signale verarbeitet.

2 Physikalischer Kern: 2D-Strahlensegmentschnitt

Der rechnerische Kern der Physik-Engine basiert auf robuster linearer Algebra, um Randkollisionen zu lösen. Eine genaue mathematische Modellierung in diesem Stadium ist für hochauflösende akustische Simulationen entscheidend. Um den Rechendurchsatz zu maximieren, was bei der Berechnung von Millionen von Strahlen zwingend erforderlich ist, verzichtet die Engine auf rechnerisch aufwändige trigonometrische Funktionen zugunsten reiner Vektormathematik.

2.1 Geometrische Definitionen

Die beiden wichtigsten geometrischen Entitäten im 2D-Simulationsraum sind der akustische Strahl und die physikalische Grenze (die eine Wand darstellt).

Der Strahl Ein Schallstrahl wird durch einen Ursprungspunkt O und einen normalisierten Richtungsvektor definiert $\vec{D}$. Jeder Punkt P entlang der Strahlenbahn kann als Funktion der skalaren Distanz t entlang dieser Richtung berechnet werden:

$$P(t) = O + t\vec{D}$$

wobei $t \geq 0$ sicherstellt, dass der Strahl vom Emitter aus im Raum weitergeht.

Die Wand (Die Strecke) Eine Wand wird als Liniensegment dargestellt, das durch zwei Endpunkte, A und B , definiert ist. Jeder Punkt W auf diesem Segment wird durch Interpolation zwischen den Endpunkten mit einem skalaren Parameter u gefunden. Definiert man den Wandvektor als $\vec{V} = B - A$, ergibt sich die parametrische Gleichung durch:

$$W(u) = A + u\vec{V}$$

Damit ein Punkt strikt innerhalb der physikalischen Grenzen des Wandsegments liegt, muss die Bedingung $0 \leq u \leq 1$ erfüllt sein.

2.2 Schnittpunktberechnung

Um den Schnittpunkt zu bestimmen, werden die Strahlen- und Wandgleichungen gleichgesetzt:

$$O + t\vec{D} = A + u\vec{V}$$

Dies bildet ein System linearer Gleichungen, um t (Ausbreitungsdistanz) und u (Segmentschnittparameter) zu lösen. Durch die Definition eines Vektors $\vec{E} =$

$A - O$, der vom Ursprung des Strahls zum Ausgangspunkt der Wand zeigt, ordnet sich die Gleichung um:

$$t\vec{D} - u\vec{V} = \vec{E}$$

Im zweidimensionalen Raum kann dieses System effizient mit dem 2D-Analogen des Kreuzprodukts gelöst werden (oft als Perp-Dot-Produkt bezeichnet). Für zwei beliebige 2D-Vektoren $\vec{v}_1 = (x_1, y_1)$ und $\vec{v}_2 = (x_2, y_2)$ ergibt sich eine skalare Determinante: $\vec{v}_1 \times \vec{v}_2 = x_1 y_2 - y_1 x_2$.

Durch die Definition der Schnittdeterminante als $\det = \vec{D} \times \vec{V}$ können die Parameter t und u isoliert und algebraisch gelöst werden:

$$t = \frac{\vec{E} \times \vec{V}}{\det}$$

$$u = \frac{\vec{E} \times \vec{D}}{\det}$$

2.3 Schnittpunkt-Validierung (Trefferbedingungen)

Eine mathematisch gültige Lösung des linearen Systems garantiert nicht automatisch eine physikalische Kollision. Die folgenden Einschränkungen müssen überprüft werden, um eine gültige akustische Reflexion zu bestätigen:

- **Parallelität:** Wenn $\det \approx 0$, sind der Strahl und die Wand perfekt parallel und schneiden sich niemals. In der Implementierung ist ein kleiner Epsilon-Schwellenwert anstelle einer exakten Nullprüfung verwendet, um Gleitkommaraunauigkeiten zu vermeiden:

```
if det.abs() < 1e-6 {
    return None;
}
```

- **Richtungsabhängigkeit:** Wenn $t < 0$, schneiden sich die mathematischen Geraden, aber die Kollision erfolgt hinter dem Aussendungspunkt des Strahls:

```
if ray_t < 0.0 {
    return None;
}
```

- **Segmentgrenzen:** Wenn $u < 0$ oder $u > 1$, schneidet der Strahl zwar die unendliche Gerade, auf der die Wand liegt, verfehlt jedoch die physische Strecke

```
if wall_u < 0.0 || wall_u > 1.0 {
    return None;
}
```

2.4 Vektorreflexion

Nach der Bestätigung eines gültigen Schnittpunkts muss die resultierende Trajektorie der Schallwelle berechnet werden. Dies erfordert den Oberflächennormalenvektor $\vec{N}$, der streng senkrecht von der Wand weg zeigt. Für einen gegebenen 2D-Wandvektor $\vec{V} = (x, y)$ ist die normierte senkrechte Normale $\vec{N} = (-y, x)$.

Die Richtung des reflektierten Strahls $\vec{R}$ wird bestimmt, indem von der Richtung des einfallenden Strahls $\vec{D}$ das Doppelte der Projektion dieser Richtung auf die Oberflächennormale subtrahiert wird.

$$\vec{R} = \vec{D} - 2(\vec{D} \cdot \vec{N})\vec{N}$$

Innerhalb der Rust-Codebasis wird diese Berechnung der Vektorreflexion durch die `reflect` Methode optimiert, die vom `glam`-Crate bereitgestellt wird und Single Instruction, Multiple Data (SIMD)-Instruktionen nutzt, um diese Operationen mit minimalem Overhead durchzuführen.

3 Hochleistungs-Nebenläufigkeit und Datenparallelität

Ein wesentlicher Vorteil von Rust im High-Performance-Computing sind seine strengen Garantien zur Kompilierzeit gegen Data-Races. Die Verteilung einer rechenintensiven, iterativen Physiksleife auf Multi-Core-Prozessorarchitekturen erfordert in traditionellen Programmiersprachen oft komplexe Thread-Sperrlogiken (z.B. Mutexe), um Speicherbeschädigungen zu verhindern. In dieser Engine wird dieser Overhead vermieden durch ein funktionales, datenparalleles Paradigma unter Verwendung des `rayon`-Crates.

3.1 Architektonischer Wandel: Von Shared Mutation zu thread-lokalem-Folding

Bei einer standardmäßigen Single-Thread-Ausführung hängen sequenzielle Iterationen die berechneten Schnittpunktdateien (z.B. `delay` und `pressure` werte) an ein gemeinsames dynamisches Array an. Diese Schleife auf zahlreiche Threads aufzuteilen, würde zur gleichzeitigen Schreibversuchen auf dieselben Speicheradressen führen, was unmittelbare Data Races zur Folge hätte. Anstatt diese gemeinsam genutzt Arrays in rechenintensive, threadsichere Sperren (Locks) zu hüllen, nutzt die Architektur das Trait `into_par_iter()` von `rayon` in Kombination mit einer `fold`- und `reduce`- Pipeline.

Dieser Ansatz teilt die Arbeitslast sicher auf alle verfügbaren CPU-Kerne auf. Anstatt zu versuchen, in ein globales Array zu schreiben oder Speicher für jeden Strahl dynamisch zu allokalieren, verwendet die Engine die `fold`-Operation, um einen privaten, vorab allokalieren, thread-lokalen Puffer für jeden Kern bereitzustellen.

Jeder Thread simuliert unabhängig seine zugewiesene Teilmenge an Strahlen und hängt die Schnittpunktergebnisse sequenziell in seinen eigenen lokalen Speicherbereich an, ohne Sperren (lock-free). Sobald ein Thread seine Arbeitslast abgeschlossen hat, wird die **reduce**-Phase aktiviert, welche diese isolierten thread-lokalen Puffer sicher und effizient zu den endgültigen, vereinheitlichten Arrays zusammenführt. Dieser funktionale Ansatz garantiert die Datensicherheit und maximiert gleichzeitig die Hardwareauslastung.

```
use rayon::prelude::*;

fn run_simulation_parallel(
    config: &SimulationConfig,
    walls: &Vec<Wall>
) -> (Vec<f32>, Vec<f32>) {
    let (delays_par, pressures_par, _) = (0..config.rays_to_cast)
        .into_par_iter()
        // 1. Falten, Split
        .fold(|| {
            let local_delays = Vec::with_capacity(CAPACITY);
            let local_pressures = Vec::with_capacity(CAPACITY);
            let temp_ray_buffer = Vec::with_capacity(CAP);

            (local_delays, local_pressures, temp_ray_buffer)
        },
        |mut local_buf, _| {
            let fresh_ray = generate_initial_ray(config);
            simulate_single_ray(fresh_ray, walls, config, &mut local_buf.2);
            for (delay, pressure) in &local_buf.2 {
                local_buf.0.push(*delay);
                local_buf.1.push(*pressure);
            }
            local_buf
        })
        // 2. Reduzieren, Global Merge
        .reduce(
            || (Vec::new(), Vec::new(), Vec::new()),
            |mut a, mut b| {
                a.0.append(&mut b.0);
                a.1.append(&mut b.1);
                a
            }
        );
    (delays_par, pressures_par)
}
```

4 Leistungsoptimierungen und Profiling

Benchmarking the compiled binary (utilizing Rust’s `--release` flag for maximum LLVM optimization) revealed significant bottlenecks and subsequent optimization opportunities within the parallelized architecture.

Das Benchmarking der kompilierten Binärdatei (unter Verwendung des `--release` flag und dem `let var = Instant::now(); timer`) deckte signifikante Engpässe und darauffolgende Optimierungsmöglichkeiten innerhalb der parallelisierten Architektur auf.

4.1 Reduzierung des Speicherallokations-Overheads: `flat_map` vs. `fold`

Während die anfängliche parallele Implementierung den `flat_map`-iterator von `rayon` nutzt, offenbarte die Skalierung der Simulation auf 100.000.000 Strahlen einen signifikanten Leistungsengpass bezüglich der Speicherallokation. Die `flat_map`-Operation allozierte dynamisch einen neuen `Vec` für die Rückgabedaten jedes einzelnen Strahls. Bei der Verteilung auf alle CPU-Kerne führte dies zu Millionen von flüchtigen Heap-Allokationen pro Sekunde. Das System erlebte Memory Thrashing; die CPU-Auslastung fiel auf 0% der Hauptspeicher war ausgelastet, und das Betriebssystem entzog dem Ausführungs-Thread die Ressourcen, was zum Stillstand der Simulation führte.

Um diesen Allokations-Overhead zu beheben, wurde die parallele Pipeline refaktorisiert, um das `fold`-Muster zu verwenden. Anstatt Speicher pro Strahl zu allokalieren, stellt die `fold`-Operation exakt einen thread-lokalen Puffer (initialisiert über `Vec::with_capacity();`) pro CPU-Kern bereit. Die Kerne akkumulieren ihre lokalen Ergebnisse und führen diese erst nach Abschluss des Threads zusammen. Diese architektonische Änderung reduzierte die Heap-Allokationen um mehrere Größenordnungen und ermöglichte es dem System, 100.000.000 Strahlen effizient zu verarbeiten, ohne die Speicherressourcen zu erschöpfen.

4.2 Benchmarking Ergebnisse und I/O Overhead

Empirische Tests verdeutlichen die Wirksamkeit der optimierten Parallelisierung sowie die verborgenen Kosten von System-I/O und Thread-Initialisierung.

- **Paralleler Speedup:** Bei einer moderaten Arbeitslast wurde die Single-Thread-Simulation in 11,79 Sekunden abgeschlossen. Die durch `rayon` optimierte parallele Simulation bewältigt dieselbe Arbeitslast in 2,56 Sekunden, was einen $4,60\times$ Speedup auf der Test-Hardware ergab.
- **Thread-Pool-Kaltstart:** Bei extrem kleinen Arbeitslasten wies `rayon` einen Initialisierung-Overhead auf. In einem Benchmark mit wenigen Strahlen wurde die Single-Thread-Ausführung in 0,14 Sekunden abgeschlossen, während die parallele Ausführung 0,34 Sekunden in Anspruch nahm. Dies deutet darauf hin, dass die Kosten für die Initialisierung des Thread-Pools die Recheneinsparungen bei trivialen Arbeitslasten überwiegen

- **The I/O Engpass:** Das Profiling der End-to-End-Ausführungszeit deckte einen massiven Engpass bei der Datenserialisierung auf. In einem Test mit 10.000.000 Strahlen wurde die interne Simulation schnell ausgeführt(4, 72s Single-Thread vs. 0, 64s Parallel). Die anschließende Serialisierung und das Schreiben des Datensatzes dominierten jedoch die End-to-End-Laufzeit(37, 40s vs. 33, 24s), was die Notwendigkeit eines RAM-gestützten `BufWriter` unterstreicht.